

# Homework 6 — Magic Academy TCP Server

## GTU CSE344 System Programming

Dicle Coban

May 2026

### Title Page

|                        |                                                |
|------------------------|------------------------------------------------|
| <b>Course</b>          | CSE344 — System Programming                    |
| <b>Homework</b>        | Homework 6 — Magic Academy TCP Server          |
| <b>Student</b>         | Dicle Coban                                    |
| <b>Student ID</b>      | <i>220104004088</i>                            |
| <b>Language</b>        | C, POSIX sockets, <code>select()</code> , C11  |
| <b>Target Platform</b> | Debian 12, 64-bit POSIX-compatible environment |

# Contents

|           |                                                                                          |           |
|-----------|------------------------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                                                                      | <b>4</b>  |
| <b>2</b>  | <b>System Design</b>                                                                     | <b>4</b>  |
| 2.1       | High-Level Architecture . . . . .                                                        | 4         |
| 2.2       | Single-Threaded Multiplexing . . . . .                                                   | 4         |
| 2.3       | Connection Capacity . . . . .                                                            | 5         |
| <b>3</b>  | <b>Data Structures</b>                                                                   | <b>5</b>  |
| 3.1       | Ingredient Table . . . . .                                                               | 5         |
| 3.2       | Client State . . . . .                                                                   | 5         |
| <b>4</b>  | <b>Protocol Handling</b>                                                                 | <b>6</b>  |
| 4.1       | Enrollment State . . . . .                                                               | 6         |
| 4.2       | Wizard Commands . . . . .                                                                | 6         |
| 4.3       | Professor Commands . . . . .                                                             | 7         |
| 4.4       | Ingredient Update Rules . . . . .                                                        | 7         |
| <b>5</b>  | <b>Partial Read and Partial Write Handling</b>                                           | <b>7</b>  |
| <b>6</b>  | <b>Idle Timeout Design</b>                                                               | <b>8</b>  |
| <b>7</b>  | <b>Signal Handling and Shutdown</b>                                                      | <b>8</b>  |
| 7.1       | Server Ctrl+C . . . . .                                                                  | 8         |
| 7.2       | Client Ctrl+C . . . . .                                                                  | 9         |
| <b>8</b>  | <b>Output and Logging</b>                                                                | <b>9</b>  |
| <b>9</b>  | <b>Testing Strategy</b>                                                                  | <b>9</b>  |
| <b>10</b> | <b>Mandatory Test Scenarios</b>                                                          | <b>10</b> |
| 10.1      | Scenario 1 — Enrollment, Duplicate Names, Authorization, Unknown Com-<br>mands . . . . . | 10        |
| 10.2      | Scenario 2 — Ingredient and Spellbook Updates . . . . .                                  | 12        |
| 10.3      | Scenario 3 — Capacity Enforcement and Slot Recovery . . . . .                            | 14        |
| 10.4      | Scenario 4 — Idle Timeout and Server Shutdown . . . . .                                  | 17        |
| <b>11</b> | <b>Challenges and Solutions</b>                                                          | <b>21</b> |
| 11.1      | Challenge 1 — TCP Message Boundaries . . . . .                                           | 21        |

|                                                       |           |
|-------------------------------------------------------|-----------|
| 11.2 Challenge 2 — Timeout Without Threads . . . . .  | 21        |
| 11.3 Challenge 3 — Clean Shutdown . . . . .           | 21        |
| 11.4 Challenge 4 — Role-Based Authorization . . . . . | 21        |
| <b>12 Conclusion</b>                                  | <b>21</b> |

# 1 Introduction

This homework implements a single-threaded Magic Academy server and two TCP client programs in C. The three executables are:

- **hogwarts**: the academy server,
- **wizard**: a TCP client that submits **BREW**, **CONSUME**, and **SPELLBOOK** commands,
- **professor**: a TCP client that submits **INSPECT**, **SCROLL**, and **ROSTER** commands.

The main purpose of the assignment is to manage many TCP clients without using a thread-per-client architecture. The server keeps all client sockets in one `select()` loop, stores per-client state, enforces a maximum connection capacity, disconnects idle clients after a configurable timeout, and handles graceful shutdown on **SIGINT**.

## 2 System Design

### 2.1 High-Level Architecture

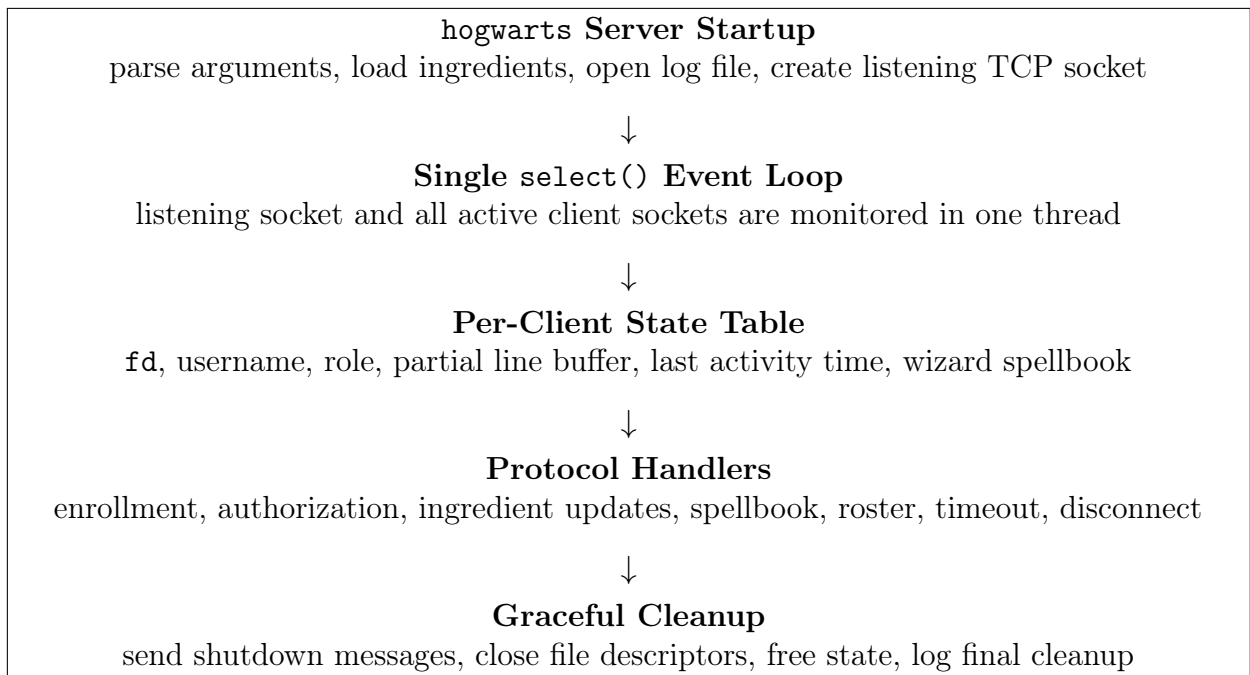


Figure 1: Overall single-threaded TCP server architecture.

### 2.2 Single-Threaded Multiplexing

The server never creates one thread per client. Instead, it builds an `fd_set` before every loop iteration. The listening socket is added so that new clients can be accepted, and every active client socket is added so that incoming commands can be read.

```

FD_ZERO(&readfds);
FD_SET(listen_fd, &readfds);

for (int i = 0; i < max_clients; ++i) {
    if (clients[i].fd >= 0) {
        FD_SET(clients[i].fd, &readfds);
    }
}

ready = select(maxfd + 1, &readfds, NULL, NULL, &timeout);

```

When `select()` reports the listening socket as readable, the server calls `accept()`. When a client socket is readable, the server calls `recv()`, appends the bytes to that client's line buffer, and processes every complete newline-terminated command.

## 2.3 Connection Capacity

The server keeps exactly `max_clients` client slots. If all slots are occupied, a new connection is still accepted first. Then the server sends `ERR HOGWARTS_FULL`, logs the rejection, and closes the new file descriptor.

```

if (slot < 0) {
    send_line(fd, "ERR_HOGWARTS_FULL");
    log_event("REJECTED_fd=%d_ip=%s_reason=HOGWARTS_FULL", fd, ip);
    close(fd);
    return;
}

```

This is important because the client receives a meaningful protocol error instead of only observing a silent connection failure.

# 3 Data Structures

## 3.1 Ingredient Table

Ingredients are stored in a fixed-size array. Invalid lines in the input file are skipped silently.

```

typedef struct {
    char name[17];
    int qty;
} ingredient_t;

```

The ingredient name rule is uppercase letters, digits, and underscores, with at most 16 characters. Quantities are positive integers during input loading and non-negative integers during runtime updates.

## 3.2 Client State

Each active TCP client has a state record:

```
typedef struct {
    int fd;
    char username[32];
    char type[16];
    char line_buf[MAX_LINE + 1];
    size_t line_len;
    int discard_long_line;
    time_t last_active;
    int *spellbook;
} client_t;
```

| Field              | Purpose                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------|
| fd                 | TCP socket file descriptor. A value of -1 means the slot is free.                           |
| username           | Set after a successful ENROLL. Duplicate names are rejected globally.                       |
| type               | Either WIZARD or PROFESSOR. Used for authorization checks.                                  |
| line_buf, line_len | Store incomplete TCP command lines until a newline arrives.                                 |
| discard_long_line  | Used after ERR TOOLONG; bytes are ignored until the next newline.                           |
| last_active        | Timestamp of the last complete command, used for idle timeout enforcement.                  |
| spellbook          | Wizard-only ingredient inventory, indexed in the same order as the global ingredient table. |

## 4 Protocol Handling

### 4.1 Enrollment State

Every client must send ENROLL WIZARD <username> or ENROLL PROFESSOR <username> before any other command. If a command arrives before enrollment, the server responds with ERR NOT\_ENROLLED. If the username is already active, the server responds with ERR ENROLL name\_taken.

### 4.2 Wizard Commands

Wizards may use:

- BREW <ingredient> <qty>: increases the global stock and the wizard's spellbook,
- CONSUME <ingredient> <qty>: decreases the global stock and the wizard's spellbook,
- SPELLBOOK: returns the wizard's current personal inventory,
- APPARATE: disconnects cleanly.

Professor-only commands such as `INSPECT`, `SCROLL`, and `ROSTER` are rejected for wizards with `ERR UNAUTHORIZED`.

### 4.3 Professor Commands

Professors may use:

- `INSPECT <ingredient>`: reads one global ingredient quantity,
- `SCROLL`: returns the full global ingredient snapshot,
- `ROSTER`: lists connected enrolled clients,
- `APPARATE`: disconnects cleanly.

Wizard-only commands such as `BREW`, `CONSUME`, and `SPELLBOOK` are rejected for professors with `ERR UNAUTHORIZED`.

### 4.4 Ingredient Update Rules

`BREW` adds the requested positive quantity to both the global ingredient stock and the wizard's spellbook. `CONSUME` subtracts from both. The implementation prevents the global stock and the wizard spellbook value from becoming negative. If the requested quantity is unavailable, the server responds with `ERR INSUFFICIENT_INGREDIENTS`.

## 5 Partial Read and Partial Write Handling

TCP is a byte stream. It does not preserve message boundaries. A single `recv()` may contain a partial command, exactly one command, or multiple commands. To handle this correctly, each client owns a line buffer. Incoming bytes are appended until a newline is found. Each completed line is parsed as one protocol command.

```
if (ch == '\n') {
    line[c->line_len] = '\0';
    c->line_len = 0;
    process_command(index, line);
} else {
    c->line_buf[c->line_len++] = ch;
}
```

If a line exceeds 512 bytes, the server sends `ERR TOOLONG`, clears the current buffer, and discards the remainder of the overlong line until the next newline. This prevents one bad command from corrupting the next valid command.

For writes, the helper `send_all()` repeatedly calls `send()` until the whole response has been transmitted or an unrecoverable error occurs. This prevents truncated responses when TCP accepts only a partial write.

## 6 Idle Timeout Design

Idle timeout is implemented in the server's main loop without threads and without `sleep()`. Before every `select()` call, the server checks all clients:

1. compute `elapsed = now - last_active`,
2. if `elapsed >= TIMEOUT`, send `TIMEOUT DISCONNECT`,
3. log `TIMEOUT username=X fd=N elapsed=Ns`,
4. close the socket and free the state,
5. compute the minimum remaining timeout among clients still connected,
6. pass that minimum remaining time to `select()`.

```
disconnect_expired_clients(now);
compute_select_timeout(&tv, time(NULL));
ready = select(maxfd + 1, &readfds, NULL, NULL, &tv);
```

This design lets active clients keep their session alive by sending commands, while inactive clients are disconnected automatically at the correct time.

## 7 Signal Handling and Shutdown

### 7.1 Server Ctrl+C

The server's `SIGINT` handler only sets a volatile `sig_atomic_t` flag. This keeps the handler async-signal-safe. The actual shutdown work happens in normal control flow after `select()` returns.

```
static volatile sig_atomic_t shutdown_requested = 0;

static void on_sigint(int signo)
{
    (void)signo;
    shutdown_requested = 1;
}
```

The server shutdown sequence is:

1. detect the `shutdown_requested` flag,
2. log `SHUTDOWN signal=SIGINT`,
3. send `SERVER SHUTDOWN` to every active TCP client,
4. close every client socket,
5. close the listening socket,
6. free dynamically allocated spellbooks and the client table,
7. log `CLEANUP_DONE clients=0`.



## 7.2 Client Ctrl+C

The wizard and professor clients also use a minimal `SIGINT` handler. Their main loop detects the flag, sends `APPARATE`, waits briefly for `OK APPARATE`, prints the disconnect reason, closes the socket, and exits.

## 8 Output and Logging

Server events are printed to `stdout` and written to the log file with timestamps. Important events include startup, client connection, enrollment, ingredient updates, professor queries, capacity rejection, timeout, disconnect, shutdown, and cleanup.

Client output lines use the required role prefix, for example:

```
[WIZARD hermione] CONNECTED server=127.0.0.1:5001
[WIZARD hermione] SENT BREW MOONSTONE 10
[WIZARD hermione] RECEIVED OK BREW MOONSTONE 10 110
[PROFESSOR dumbledore] RECEIVED OK ROSTER hermione,dumbledore
```

## 9 Testing Strategy

In addition to the mandatory manual scenarios, an automated Python test runner named `run_all_tests.py` was written. It starts real server processes on temporary local ports, creates raw TCP clients, sends protocol commands, validates exact responses, and stops the server cleanly.

The automated test suite covers:

- warning-free compilation with `make`,
- invalid command-line arguments,
- invalid ingredient file lines,
- command before enrollment,
- duplicate username rejection,
- wizard/professor authorization checks,
- all main protocol commands,
- unknown command and unknown ingredient errors,
- insufficient ingredient/spellbook checks,
- partial TCP reads,
- multiple commands arriving in one TCP packet,
- overlong line handling with `ERR TOOLONG`,

- ERR HOGWARTS\_FULL,
- idle timeout disconnect,
- slot recovery after timeout,
- server SIGINT shutdown,
- smoke tests using the real wizard and professor binaries.

```
[TEST] build_without_warnings ... PASS
[TEST] invalid_arguments ... PASS
[TEST] invalid_ingredient_lines ... PASS
[TEST] protocol_and_authorization ... PASS
[TEST] partial_multi_and_toolong ... PASS
[TEST] capacity_timeout_slot_recovery_shutdown ... PASS
[TEST] real_clients_smoke ... PASS

All tests passed.
```

## 10 Mandatory Test Scenarios

The following scenarios were run according to the homework specification. Each screenshot in the final report must be uncut and must show the command prompt, command, full terminal interaction, and program termination.

### 10.1 Scenario 1 — Enrollment, Duplicate Names, Authorization, Unknown Commands

```
./hogwarts -p 5001 -s ingredients.txt -l server.log -n 4 -t 30
./wizard 127.0.0.1 5001 hermione
./professor 127.0.0.1 5001 dumbledore
./wizard 127.0.0.1 5001 hermione
```

This scenario verifies duplicate username rejection, command-before-enroll handling, unknown command handling, professor rejection for BREW/CONSUME, wizard rejection for INSPECT/SCROLL, and graceful Ctrl+C disconnects.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./hogwarts -p 5001 -s ingredients.txt -l server.log -n 4 -t 30
[2026-05-19 19:14:52] [SERVER] SERVER_STARTED port=5001 max_clients=4 timeout=30 ingredients=5
Hogwarts is ready. Port: 5001 | Max Clients: 4 | Timeout: 30s
[2026-05-19 19:14:58] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 19:14:58] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-19 19:15:38] [SERVER] TIMEOUT username=hermione fd=5 elapsed=30s
[2026-05-19 19:15:38] [SERVER] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-19 19:16:00] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 19:16:00] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=5
[2026-05-19 19:16:42] [SERVER] TIMEOUT username=dumbledore fd=5 elapsed=30s
[2026-05-19 19:16:42] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=timeout
[2026-05-19 19:17:24] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 19:17:24] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-19 19:17:31] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-19 19:17:54] [SERVER] TIMEOUT username=hermione fd=5 elapsed=30s
[2026-05-19 19:17:54] [SERVER] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-19 19:18:01] [SERVER] TIMEOUT username=unknown fd=6 elapsed=30s
[2026-05-19 19:18:01] [SERVER] CLIENT_DISCONNECTED username=unknown reason=timeout
[2026-05-19 19:18:42] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 19:19:18] [SERVER] TIMEOUT username=unknown fd=5 elapsed=30s
[2026-05-19 19:19:18] [SERVER] CLIENT_DISCONNECTED username=unknown reason=timeout

```

Figure 2: Scenario 1 server terminal: Server log output for duplicate username, authorization errors, unknown command, and disconnects.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./wizard 127.0.0.1 5001 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5001
[WIZARD hermione] SENT ENROLL WIZARD hermione
[WIZARD hermione] RECEIVED OK ENROLL hermione
FLY BROOMSTICK
[WIZARD hermione] SENT FLY BROOMSTICK
[WIZARD hermione] RECEIVED ERR UNKNOWN FLY
INSPECT MOONSTONE
[WIZARD hermione] SENT INSPECT MOONSTONE
[WIZARD hermione] RECEIVED ERR UNAUTHORIZED
SCROLL
[WIZARD hermione] SENT SCROLL
[WIZARD hermione] RECEIVED ERR UNAUTHORIZED
[WIZARD hermione] RECEIVED TIMEOUT DISCONNECT
[WIZARD hermione] DISCONNECTED reason=timeout

```

Figure 3: Scenario 1 wizard terminal: Wizard commands and responses for Hermione.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./professor 127.0.0.1 5001 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5001
[PROFESSOR dumbledore] SENT ENROLL PROFESSOR dumbledore
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
BREW MOONSTONE 5
[PROFESSOR dumbledore] SENT BREW MOONSTONE 5
[PROFESSOR dumbledore] RECEIVED ERR UNAUTHORIZED
CONSUME MOONSTONE 5
[PROFESSOR dumbledore] SENT CONSUME MOONSTONE 5
[PROFESSOR dumbledore] RECEIVED ERR UNAUTHORIZED

```

Figure 4: Scenario 1 professor terminal: Professor commands and responses for Dumbledore.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./wizard 127.0.0.1 5001 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5001
[WIZARD hermione] SENT ENROLL WIZARD hermione
[WIZARD hermione] RECEIVED ERR ENROLL name_taken

```

Figure 5: Scenario 1 duplicate username terminal: Second Hermione connection receiving ERR ENROLL name\_taken.

```

❖ diclesaracoban@Dicle-MacBook-Pro HW6 % nc 127.0.0.1 5001
BREW MOONSTONE 1
ERR NOT_ENROLLED

```

Figure 6: Scenario 1 not enrolled terminal: A client sending a command before ENROLL.

## 10.2 Scenario 2 — Ingredient and Spellbook Updates

```

./hogwarts -p 5001 -s ingredients.txt -l server.log -n 4 -t 30
./wizard 127.0.0.1 5001 harry
./professor 127.0.0.1 5001 mcgonagall

```

This scenario verifies BREW, CONSUME, SPELLBOOK, unknown ingredient handling, INSPECT, SCROLL, ROSTER, and graceful exits.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % rm -f server.log
./hogwarts -p 5001 -s ingredients.txt -l server.log -n 4 -t 30
[2026-05-19 19:24:02] [SERVER] SERVER_STARTED port=5001 max_clients=4 timeout=30 ingredients=5
Hogwarts is ready. Port: 5001 | Max Clients: 4 | Timeout: 30s
[2026-05-19 19:24:07] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 19:24:07] [SERVER] ENROLL username=harry type=WIZARD fd=5
[2026-05-19 19:24:13] [SERVER] BREW wizard=harry ingredient=MOONSTONE qty=10 old_qty=100 new_qty=110
[2026-05-19 19:24:24] [SERVER] BREW wizard=harry ingredient=UNICORN_HAIR qty=15 old_qty=20 new_qty=35
[2026-05-19 19:24:32] [SERVER] CONSUME wizard=harry ingredient=MOONSTONE qty=5 old_qty=110 new_qty=105
[2026-05-19 19:25:17] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-19 19:25:17] [SERVER] TIMEOUT username=harry fd=5 elapsed=30s
[2026-05-19 19:25:17] [SERVER] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-19 19:25:17] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-19 19:25:25] [SERVER] INSPECT professor=mcgonagall ingredient=DRAGON_SCALE qty=50
[2026-05-19 19:25:28] [SERVER] SCROLL professor=mcgonagall ingredients=5
[2026-05-19 19:25:30] [SERVER] ROSTER professor=mcgonagall clients=1
[2026-05-19 19:26:01] [SERVER] CLIENT_DISCONNECTED username=mcgonagall reason=APPARATE

```

Figure 7: Scenario 2 server terminal: Server output for ingredient updates, professor queries, and disconnects.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./wizard 127.0.0.1 5001 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5001
[WIZARD harry] SENT ENROLL WIZARD harry
[WIZARD harry] RECEIVED OK ENROLL harry
BREW MOONSTONE 10
[WIZARD harry] SENT BREW MOONSTONE 10
[WIZARD harry] RECEIVED OK BREW MOONSTONE 10 110
SPELLBOOK
[WIZARD harry] SENT SPELLBOOK
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10
BREW UNICORN_HAIR 15
[WIZARD harry] SENT BREW UNICORN_HAIR 15
[WIZARD harry] RECEIVED OK BREW UNICORN_HAIR 15 35
SPELLBOOK
[WIZARD harry] SENT SPELLBOOK
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10,UNICORN_HAIR:15
CONSUME MOONSTONE 5
[WIZARD harry] SENT CONSUME MOONSTONE 5
[WIZARD harry] RECEIVED OK CONSUME MOONSTONE 5 105
CONSUME UNICORN_HAIR 20
[WIZARD harry] SENT CONSUME UNICORN_HAIR 20
[WIZARD harry] RECEIVED ERR INSUFFICIENT_INGREDIENTS
BREW BUTTERBEER 5
[WIZARD harry] SENT BREW BUTTERBEER 5
[WIZARD harry] RECEIVED ERR UNKNOWN_INGREDIENT

```

Figure 8: Scenario 2 wizard terminal: Harry's BREW, CONSUME, and SPELLBOOK responses.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./professor 127.0.0.1 5001 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5001
[PROFESSOR mcgonagall] SENT ENROLL PROFESSOR mcgonagall
[PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
INSPECT DRAGON_SCALE
[PROFESSOR mcgonagall] SENT INSPECT DRAGON_SCALE
[PROFESSOR mcgonagall] RECEIVED OK INSPECT DRAGON_SCALE 50
SCROLL
[PROFESSOR mcgonagall] SENT SCROLL
[PROFESSOR mcgonagall] RECEIVED OK SCROLL MOONSTONE:105,DRAGON_SCALE:50,PHOENIX_FEATHER:30,MANDRAKE_ROOT:75,UNICORN_HAIR:35
ROSTER
[PROFESSOR mcgonagall] SENT ROSTER
[PROFESSOR mcgonagall] RECEIVED OK ROSTER mcgonagall
INSPECT BUTTERBEER
[PROFESSOR mcgonagall] SENT INSPECT BUTTERBEER
[PROFESSOR mcgonagall] RECEIVED ERR UNKNOWN_INGREDIENT

```

Figure 9: Scenario 2 professor terminal: Professor queries and roster output.

### 10.3 Scenario 3 — Capacity Enforcement and Slot Recovery

```

./hogwarts -p 5001 -s ingredients.txt -l server.log -n 3 -t 60
./wizard 127.0.0.1 5001 harry
./wizard 127.0.0.1 5001 ron
./professor 127.0.0.1 5001 dumbledore
./wizard 127.0.0.1 5001 voldemort
./professor 127.0.0.1 5001 snape

```

This scenario verifies that the fourth and fifth simultaneous clients receive `ERR HOGWARTS_FULL`. After Harry disconnects, Neville can connect successfully, proving that the server recovers client slots after disconnect.



```

diclesaracoban@Dicle-MacBook-Pro HW6 % rm -f server.log
./hogwarts -p 5001 -s ingredients.txt -l server.log -n 3 -t 60
[2026-05-19 19:26:41] [SERVER] SERVER_STARTED port=5001 max_clients=3 timeout=60 ingredients=5
Hogwarts is ready. Port: 5001 | Max Clients: 3 | Timeout: 60s
[2026-05-19 19:26:49] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 19:26:49] [SERVER] ENROLL username=harry type=WIZARD fd=5
[2026-05-19 19:26:56] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-19 19:26:56] [SERVER] ENROLL username=ron type=WIZARD fd=6
[2026-05-19 19:27:09] [SERVER] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-19 19:27:09] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=7
[2026-05-19 19:27:19] [SERVER] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-19 19:27:49] [SERVER] TIMEOUT username=harry fd=5 elapsed=60s
[2026-05-19 19:27:49] [SERVER] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-19 19:27:56] [SERVER] TIMEOUT username=ron fd=6 elapsed=60s
[2026-05-19 19:27:56] [SERVER] CLIENT_DISCONNECTED username=ron reason=timeout
[2026-05-19 19:28:09] [SERVER] TIMEOUT username=dumbledore fd=7 elapsed=60s
[2026-05-19 19:28:09] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=timeout
[2026-05-19 19:29:37] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 19:29:37] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=5
[2026-05-19 19:29:40] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-19 19:29:40] [SERVER] ENROLL username=neville type=WIZARD fd=6
[2026-05-19 19:29:45] [SERVER] BREW wizard=neville ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-19 19:30:03] [SERVER] ROSTER professor=dumbledore clients=2
[2026-05-19 19:30:27] [SERVER] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-19 19:30:27] [SERVER] ENROLL username=ron type=WIZARD fd=7
[2026-05-19 19:30:31] [SERVER] ROSTER professor=dumbledore clients=3
[2026-05-19 19:30:41] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-19 19:30:44] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 19:30:44] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=5
[2026-05-19 19:30:45] [SERVER] TIMEOUT username=neville fd=6 elapsed=60s
[2026-05-19 19:30:45] [SERVER] CLIENT_DISCONNECTED username=neville reason=timeout
[2026-05-19 19:30:46] [SERVER] ROSTER professor=dumbledore clients=2
[2026-05-19 19:30:55] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-19 19:30:59] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 19:30:59] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=5
[2026-05-19 19:31:01] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-19 19:31:01] [SERVER] ENROLL username=neville type=WIZARD fd=6
[2026-05-19 19:31:07] [SERVER] BREW wizard=neville ingredient=MOONSTONE qty=5 old_qty=105 new_qty=110
[2026-05-19 19:31:11] [SERVER] ROSTER professor=dumbledore clients=3
[2026-05-19 19:31:27] [SERVER] TIMEOUT username=ron fd=7 elapsed=60s
[2026-05-19 19:31:27] [SERVER] CLIENT_DISCONNECTED username=ron reason=timeout
[2026-05-19 19:32:07] [SERVER] TIMEOUT username=neville fd=6 elapsed=60s
[2026-05-19 19:32:07] [SERVER] CLIENT_DISCONNECTED username=neville reason=timeout
[2026-05-19 19:32:11] [SERVER] TIMEOUT username=dumbledore fd=5 elapsed=60s
[2026-05-19 19:32:11] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=timeout

```

Figure 10: Scenario 3 server terminal: Server capacity rejection, slot recovery, roster, and disconnect logs.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./wizard 127.0.0.1 5001 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5001
[WIZARD harry] SENT ENROLL WIZARD harry
[WIZARD harry] RECEIVED OK ENROLL harry

```

Figure 11: Scenario 3 Harry terminal: Harry connects and later disconnects to free a slot.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./wizard 127.0.0.1 5001 ron
[WIZARD ron] CONNECTED server=127.0.0.1:5001
[WIZARD ron] SENT ENROLL WIZARD ron
[WIZARD ron] RECEIVED OK ENROLL ron

```

Figure 12: Scenario 3 Ron terminal: Ron remains connected until graceful exit.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./professor 127.0.0.1 5001 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5001
[PROFESSOR dumbledore] SENT ENROLL PROFESSOR dumbledore
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
ROSTER
[PROFESSOR dumbledore] SENT ROSTER
[PROFESSOR dumbledore] RECEIVED OK ROSTER dumbledore,neville,ron

```

Figure 13: Scenario 3 Dumbledore terminal: Dumbledore roster query after Neville joins.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./wizard 127.0.0.1 5001 voldemort
[WIZARD voldemort] CONNECTED server=127.0.0.1:5001
[WIZARD voldemort] SENT ENROLL WIZARD voldemort
[WIZARD voldemort] RECEIVED ERR HOGWARTS_FULL
recv: Connection reset by peer

```

Figure 14: Scenario 3 rejected wizard terminal: Voldemort receives ERR HOGWARTS\_FULL when the server is full.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./professor 127.0.0.1 5001 Snape
[PROFESSOR Snape] CONNECTED server=127.0.0.1:5001
[PROFESSOR Snape] SENT ENROLL PROFESSOR Snape
[PROFESSOR Snape] RECEIVED ERR HOGWARTS_FULL
recv: Connection reset by peer
diclesaracoban@Dicle-MacBook-Pro HW6 %

```

Figure 15: Scenario 3 rejected professor terminal: Snape receives ERR HOGWARTS\_FULL as the fifth simultaneous client.



```
diclesaracoban@Dicle-MacBook-Pro HW6 % ./wizard 127.0.0.1 5001 neville
[WIZARD neville] CONNECTED server=127.0.0.1:5001
[WIZARD neville] SENT ENROLL WIZARD neville
[WIZARD neville] RECEIVED OK ENROLL neville
BREW MOONSTONE 5
[WIZARD neville] SENT BREW MOONSTONE 5
[WIZARD neville] RECEIVED OK BREW MOONSTONE 5 105
```

Figure 16: Scenario 3 Neville terminal: Neville connects after a slot is recovered and sends BREW.

## 10.4 Scenario 4 — Idle Timeout and Server Shutdown

```
./hogwarts -p 5001 -s ingredients.txt -l server.log -n 6 -t 10
./wizard 127.0.0.1 5001 hermione
./wizard 127.0.0.1 5001 harry
./wizard 127.0.0.1 5001 ron
./professor 127.0.0.1 5001 dumbledore
./professor 127.0.0.1 5001 mcgonagall
```

This scenario verifies idle timeout behavior. Hermione sends commands periodically, so her timer is refreshed. Harry sends nothing and is disconnected with `TIMEOUT DISCONNECT`. Later, new clients connect, professor queries are executed, and the server is stopped with `Ctrl+C`. The final checks are:

```
ps aux | grep hogwarts
cat server.log
```

```

diclesaracoban@Dicle-MacBook-Pro HW6 % rm -f server.log
./hogwarts -p 5001 -s ingredients.txt -l server.log -n 6 -t 10
[2026-05-19 23:31:51] [SERVER] SERVER_STARTED port=5001 max_clients=6 timeout=10 ingredients=5
Hogwarts is ready. Port: 5001 | Max Clients: 6 | Timeout: 10s
[2026-05-19 23:31:53] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 23:31:53] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-19 23:31:57] [SERVER] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-19 23:32:07] [SERVER] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[2026-05-19 23:32:10] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-19 23:32:10] [SERVER] ENROLL username=harry type=WIZARD fd=6
[2026-05-19 23:32:20] [SERVER] TIMEOUT username=harry fd=6 elapsed=10s
[2026-05-19 23:32:20] [SERVER] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-19 23:32:31] [SERVER] TIMEOUT username=hermione fd=5 elapsed=10s
[2026-05-19 23:32:31] [SERVER] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-19 23:33:34] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 23:33:34] [SERVER] ENROLL username=ron type=WIZARD fd=5
[2026-05-19 23:33:40] [SERVER] BREW wizard=ron ingredient=UNICORN_HAIR qty=5 old_qty=20 new_qty=25
[2026-05-19 23:33:50] [SERVER] TIMEOUT username=ron fd=5 elapsed=10s
[2026-05-19 23:33:50] [SERVER] CLIENT_DISCONNECTED username=ron reason=timeout
[2026-05-19 23:34:05] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 23:34:05] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=5
[2026-05-19 23:34:07] [SERVER] SCROLL professor=dumbledore ingredients=5
[2026-05-19 23:34:09] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-19 23:34:09] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-19 23:34:11] [SERVER] ROSTER professor=mcgonagall clients=2
[2026-05-19 23:34:17] [SERVER] TIMEOUT username=dumbledore fd=5 elapsed=10s
[2026-05-19 23:34:17] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=timeout
[2026-05-19 23:34:21] [SERVER] TIMEOUT username=mcgonagall fd=6 elapsed=10s
[2026-05-19 23:34:21] [SERVER] CLIENT_DISCONNECTED username=mcgonagall reason=timeout
^C[2026-05-19 23:35:14] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-19 23:35:14] [SERVER] CLEANUP_DONE clients=0

```

Figure 17: Scenario 4 server terminal: Server timeout, new clients, professor queries, Ctrl+C shutdown, and cleanup.

```

● diclesaracoban@Dicle-MacBook-Pro HW6 % ./wizard 127.0.0.1 5001 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5001
[WIZARD hermione] SENT ENROLL WIZARD hermione
[WIZARD hermione] RECEIVED OK ENROLL hermione
BREW MOONSTONE 5
[WIZARD hermione] SENT BREW MOONSTONE 5
[WIZARD hermione] RECEIVED OK BREW MOONSTONE 5 105
SPELLBOOK
[WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5
BREW DRAGON_SCALE 2
[WIZARD hermione] SENT BREW DRAGON_SCALE 2
[WIZARD hermione] RECEIVED OK BREW DRAGON_SCALE 2 52
SPELLBOOK
[WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5,DRAGON_SCALE:2
SPELLBOOK
[WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5,DRAGON_SCALE:2
SPELLBOOK
[WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5,DRAGON_SCALE:2
[WIZARD hermione] RECEIVED TIMEOUT DISCONNECT
[WIZARD hermione] DISCONNECTED reason=timeout

```

Figure 18: Scenario 4 Hermione terminal: Hermione remains active across timeout intervals.

```

● diclesaracoban@Dicle-MacBook-Pro HW6 % ./wizard 127.0.0.1 5001 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5001
[WIZARD harry] SENT ENROLL WIZARD harry
[WIZARD harry] RECEIVED OK ENROLL harry
[WIZARD harry] RECEIVED TIMEOUT DISCONNECT
[WIZARD harry] DISCONNECTED reason=timeout

```

Figure 19: Scenario 4 Harry terminal: Harry receives TIMEOUT DISCONNECT.

```

○ diclesaracoban@Dicle-MacBook-Pro HW6 % ./wizard 127.0.0.1 5001 ron
[WIZARD ron] CONNECTED server=127.0.0.1:5001
[WIZARD ron] SENT ENROLL WIZARD ron
[WIZARD ron] RECEIVED OK ENROLL ron
BREW UNICORN_HAIR 5
[WIZARD ron] SENT BREW UNICORN_HAIR 5
[WIZARD ron] RECEIVED OK BREW UNICORN_HAIR 5 25

```

Figure 20: Scenario 4 Ron terminal: Ron connects after timeout and sends BREW.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./professor 127.0.0.1 5001 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5001
[PROFESSOR dumbledore] SENT ENROLL PROFESSOR dumbledore
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
SCROLL
[PROFESSOR dumbledore] SENT SCROLL
[PROFESSOR dumbledore] RECEIVED OK SCROLL MOONSTONE:105,DRAGON_SCALE:52,PHOENIX_FEATHER:30,MANDRAKE_ROOT:75,UNICORN_HAIR:25
[PROFESSOR dumbledore] RECEIVED TIMEOUT DISCONNECT
[PROFESSOR dumbledore] DISCONNECTED reason=timeout
diclesaracoban@Dicle-MacBook-Pro HW6 %

```

Figure 21: Scenario 4 dumbledore terminal: Dumbledore runs SCROLL.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ./professor 127.0.0.1 5001 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5001
[PROFESSOR mcgonagall] SENT ENROLL PROFESSOR mcgonagall
[PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
ROSTER
[PROFESSOR mcgonagall] SENT ROSTER
[PROFESSOR mcgonagall] RECEIVED OK ROSTER dumbledore,mcgonagall
[PROFESSOR mcgonagall] RECEIVED TIMEOUT DISCONNECT
[PROFESSOR mcgonagall] DISCONNECTED reason=timeout
diclesaracoban@Dicle-MacBook-Pro HW6 %

```

Figure 22: Scenario 4 mcgonagall terminal: McGonagall runs ROSTER.

```

diclesaracoban@Dicle-MacBook-Pro HW6 % ps aux | grep hogwarts
cat server.log
diclesaracoban  20595  0.0  0.0 441879264  1104 s006  U+   11:35PM   0:00.00 grep hogwarts
[2026-05-19 23:31:51] [SERVER] SERVER_STARTED port=5001 max_clients=6 timeout=10 ingredients=5
[2026-05-19 23:31:53] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 23:31:53] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-19 23:31:57] [SERVER] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-19 23:32:07] [SERVER] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[2026-05-19 23:32:10] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-19 23:32:10] [SERVER] ENROLL username=harry type=WIZARD fd=6
[2026-05-19 23:32:20] [SERVER] TIMEOUT username=harry fd=6 elapsed=10s
[2026-05-19 23:32:20] [SERVER] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-19 23:32:31] [SERVER] TIMEOUT username=hermione fd=5 elapsed=10s
[2026-05-19 23:32:31] [SERVER] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-19 23:33:34] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 23:33:34] [SERVER] ENROLL username=ron type=WIZARD fd=5
[2026-05-19 23:33:40] [SERVER] BREW wizard=ron ingredient=UNICORN_HAIR qty=5 old_qty=20 new_qty=25
[2026-05-19 23:33:50] [SERVER] TIMEOUT username=ron fd=5 elapsed=10s
[2026-05-19 23:33:50] [SERVER] CLIENT_DISCONNECTED username=ron reason=timeout
[2026-05-19 23:34:05] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-19 23:34:05] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=5
[2026-05-19 23:34:07] [SERVER] SCROLL professor=dumbledore ingredients=5
[2026-05-19 23:34:09] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-19 23:34:09] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-19 23:34:11] [SERVER] ROSTER professor=mcgonagall clients=2
[2026-05-19 23:34:17] [SERVER] TIMEOUT username=dumbledore fd=5 elapsed=10s
[2026-05-19 23:34:17] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=timeout
[2026-05-19 23:34:21] [SERVER] TIMEOUT username=mcgonagall fd=6 elapsed=10s
[2026-05-19 23:34:21] [SERVER] CLIENT_DISCONNECTED username=mcgonagall reason=timeout
[2026-05-19 23:35:14] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-19 23:35:14] [SERVER] CLEANUP_DONE clients=0
diclesaracoban@Dicle-MacBook-Pro HW6 %

```

Figure 23: Scenario 4 process and log check: `ps aux | grep hogwarts` and final `cat server.log`.

## 11 Challenges and Solutions

### 11.1 Challenge 1 — TCP Message Boundaries

The most important networking issue was that TCP does not preserve line boundaries. A command can arrive in fragments, or multiple commands can arrive together. This was solved with a per-client line buffer. Complete commands are extracted only when a newline is received. The automated test suite confirms both fragmented commands and multiple commands in a single send.

### 11.2 Challenge 2 — Timeout Without Threads

Idle timeout could not be implemented with a separate timeout thread or with `sleep()`. The solution was to recalculate the nearest timeout before every `select()` call. Expired clients are disconnected before entering `select()`, and the next `select()` timeout is set to the minimum remaining time among active clients.

### 11.3 Challenge 3 — Clean Shutdown

Both server and clients must respond correctly to Ctrl+C. Signal handlers are intentionally minimal and only set `sig_atomic_t` flags. The normal program loop performs socket sends, file descriptor cleanup, memory freeing, and log writing. This avoids unsafe work inside the signal handler.

### 11.4 Challenge 4 — Role-Based Authorization

The server must reject commands that do not belong to the enrolled client type. The command dispatcher first checks enrollment, then checks the client role before performing command-specific work. This prevents professors from changing ingredient quantities and prevents wizards from using professor-only inspection commands.

## 12 Conclusion

The implementation satisfies the core requirements of the assignment. The server is single-threaded, uses `select()` for all TCP clients, maintains per-client state, tracks wizard spell-books, updates global ingredient quantities, enforces maximum client capacity, disconnects idle clients, handles partial reads and writes, and exits cleanly on `SIGINT`. The client programs enroll automatically, multiplex stdin and server responses, and send `APPARATE` during graceful exit.